

BMI500 Homework 3

Jasmine Zhou

Department of Computer Science, Emory University

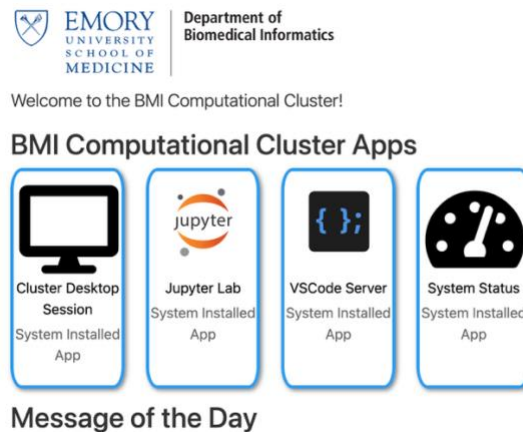
September 14, 2026

Table of Contents

<i>OddJobs Login</i>	2
<i>Code Review</i>	2
AI code review	2
Manual code review	3
<i>Coarse-grained Profiling</i>	4
<i>Instrumental Profiling</i>	6
Per-section Runtime by Dataset	7
<i>Fine-grained Profiling (cProfile)</i>	8
Console output	8
cProfile Visualizations	8
<i>SLURM Scripts</i>	10
<i>Supplemental Materials</i>	17
Vibe Coding with AI Agent.....	17
cProfiling Console Output.....	42

OddJobs Login

Via Cluster Interface



Via SSH

```
Welcome to Oddjobs-Login, the BMI Computational Cluster
job submission system!

Using the BMI Computational Cluster (super quick-start guide):

Preferred/primary method - Submit an sbatch script:
sbatch ./path/to/script.sh

Enter an interactive bash session with 4 cpu cores & 8 GB of RAM:
srun -J myInteractiveJob -c 4 --mem 8G --pty bash -i

Get help with a program (e.g. sbatch):
man sbatch

Info about cluster queues:      https://help.bmi.emory.edu/clusterqueues
Graphical sessions and jupyterlab: https://oddjobs.bmi.emory.edu

Use the 'less' command to read the below 2 files from your SSH session:
Sbatch example script:          /opt/tools/slurm_example_script.sh
Detailed general guidelines:    /opt/tools/bmi_cluster_general_notes

-> DON'T run code directly here! (Use sbatch and srun.)
-> DON'T download/upload large amounts of data! (Contact BMI-IT about
        using Datalink.)
-> DON'T copy PHI/private data to your local machine! (It stays here.)
-> DON'T use 'watch' to monitor the cluster queue. (Use 'squeue -i' if
        you really need to, but using job status emails instead is preferred.)
-> The screen program is no longer supported! (Use tmux instead.)
-> Idle tmux sessions expire! (Use sbatch for longer background jobs.)

*** Message of the Day last updated: January 7, 2026 ***

***YOUR USE OF THE BMI COMPUTATIONAL CLUSTER INDICATES YOU HAVE READ AND
AGREE TO THE TERMS OF USE LOCATED AT /opt/tools/bmi_cluster_terms_of_use
```

Code Review

AI code review

Coding assistant: CODEX (GPT-6 Astra)

Prompt: Perform a code review of matvec_multiply.py, focusing on the implementation of the functions and correctness.

Response:

The implementation is clear, and all 14 existing tests pass. I found two correctness issues in the numeric types the functions accept:

1. [P2] Accepted NumPy scalars can overflow silently — [matvec_multiply.py (line 31)](/Users/jiesimin/bmi500_HW3/matvec_multiply.py:31). `check_vector` accepts NumPy numeric scalars through `numbers.Real`, but multiplication preserves their fixed-width arithmetic. I verified that `dot_product([np.int64(2**62)], [4])` returns 0, and multiplying two `np.float32(1e30)` values returns `inf`. The overflow guard only checks Python float results. Normalize supported scalar types before arithmetic, or narrow the accepted types, and add regression tests for both cases. This also affects `matvec_multiply`.
2. [P2] Validation rejects valid large integers and fractions — [matvec_multiply.py (line 14)](/Users/jiesimin/bmi500_HW3/matvec_multiply.py:14). `math.isfinite` converts these values to floating point, which can overflow even when the input is finite. For example, `dot_product([10**400], [1])` raises `OverflowError`, although Python can compute the result exactly. Treat integers and rational numbers as finite without converting them to floats.

Two smaller improvements:

- Floating-point accuracy: The accumulation at [line 31 (line 31)](/Users/jiesimin/bmi500_HW3/matvec_multiply.py:31) loses small contributions during cancellation: `[1e16, 1.0, -1e16]` dotted with `[1.0, 1.0, 1.0]` returns 0.0 instead of 1.0. Consider compensated summation if numerical accuracy is a requirement, while preserving exact integer arithmetic.
- Benchmark timing: Use `time.perf_counter()` in [main (line 64)](/Users/jiesimin/bmi500_HW3/matvec_multiply.py:64) for elapsed time; `time.time()` can change when the system clock is adjusted.

The matrix dimension checks and reuse of `dot_product` are otherwise sound for ordinary Python integers and floats. No files were changed.

Manual code review

Partner: Zhu Xu

Overall, this is a solid implementation for completing the dot production of two vectors and computing the matrix-vector product.

There are a lot of fallbacks and sanity checks, like the input must be a real number, must be a list/tuple, and must be finite.

However, there are still some areas for improvement. Docstring on the public function can be added for clarity, and the left-over comments can be deleted.

The current implementation also rejects an empty matrix arbitrarily. `matvec_multiply([], [])` raises a `ValueError`, but `dot_product([], [])` returns 0. Consider dropping `len(A) == 0` because the matrix multiplication should be allowed to return an empty result.

There's currently a separate `def check_vector(v, name)` function that serves as sanity checks. Consider incorporating it within the existing functions so no additional function is needed beyond what's required.

Coarse-grained Profiling

PBMC3k Console Output (selected)

- Time: 23.96s for user, 1s for system
- CPU: 96%
- Memory: 865,280 kB

```
User time (seconds): 23.96
System time (seconds): 1.00
Percent of CPU this job got: 96%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:25.94
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 865280
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 2
Minor (reclaiming a frame) page faults: 163432
Voluntary context switches: 9259
Involuntary context switches: 99
Swaps: 0
File system inputs: 19848
File system outputs: 87760
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

PBMC6k Console Output (selected)

- Time: 34.02s for user, 1.35s for system
- CPU: 93%
- Memory: 887,172 kB

```
User time (seconds): 34.02
System time (seconds): 1.35
Percent of CPU this job got: 93%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:37.68
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 887172
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 2444
Minor (reclaiming a frame) page faults: 203694
Voluntary context switches: 16520
Involuntary context switches: 111
Swaps: 0
File system inputs: 581856
File system outputs: 160432
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

PBMC10k Console Output (selected)

- Time: 75.07s for user, 7.96s for system
- CPU: 97%
- Memory: 1,878,476 kB

```
User time (seconds): 75.07
System time (seconds): 7.96
Percent of CPU this job got: 97%
Elapsed (wall clock) time (h:mm:ss or m:ss): 1:24.91
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 1878476
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 2527
Minor (reclaiming a frame) page faults: 845885
Voluntary context switches: 27256
Involuntary context switches: 310
Swaps: 0
File system inputs: 1197936
File system outputs: 831672
```

```
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

Instrumental Profiling

Major bottleneck across different datasets: startup_argparse, neighbors, umap, and rank_genes_groups.

- startup_argparse: measures startup/import overhead
- neighbors: builds a k-nearest neighbor graph in the 30-dimensional PCA space,
- umap: runs UMAP on that neighbor graph
- rank_genes_groups: a Wilcoxon rank-sum test of each gene in each Louvain cluster against all other cells, to find marker genes

PBMC3k Console Output

```
=== Timing summary per section ===
Function                                Time  Percentage of total time
-----
startup_argparse                        7.179 s ( 31.8%)
read_10x_mtx                           0.154 s (  0.7%)
filter_cells_genes                      0.056 s (  0.2%)
normalize_log1p                         0.044 s (  0.2%)
highly_variable_genes                  0.058 s (  0.3%)
scale                                  0.043 s (  0.2%)
pca                                     0.312 s (  1.4%)
neighbors                              4.244 s ( 18.8%)
louvain                                0.134 s (  0.6%)
umap                                    8.332 s ( 36.9%)
write_h5ad                              0.149 s (  0.7%)
rank_genes_groups                       1.857 s (  8.2%)
TOTAL                                  22.562 s
```

PBMC6k Console Output

```
=== Timing summary per section ===
```

Function	Time	Percentage of total time

startup_argparse	7.362 s	(22.1%)
read_10x_mtx	0.211 s	(0.6%)
filter_cells_genes	0.088 s	(0.3%)
normalize_log1p	0.073 s	(0.2%)
highly_variable_genes	0.089 s	(0.3%)
scale	0.080 s	(0.2%)
pca	0.592 s	(1.8%)
neighbors	4.380 s	(13.2%)
louvain	0.374 s	(1.1%)
umap	16.114 s	(48.4%)
write_h5ad	0.268 s	(0.8%)
rank_genes_groups	3.655 s	(11.0%)
TOTAL	33.285 s	

PBMC10k Console Output

```
=== Timing summary per section ===
```

Function	Time	Percentage of total time

startup_argparse	7.298 s	(9.1%)
read_10x_mtx	8.008 s	(10.0%)
filter_cells_genes	0.427 s	(0.5%)
normalize_log1p	0.452 s	(0.6%)
highly_variable_genes	0.466 s	(0.6%)
scale	0.176 s	(0.2%)
pca	1.092 s	(1.4%)
neighbors	31.748 s	(39.6%)
louvain	0.795 s	(1.0%)
umap	14.165 s	(17.7%)
write_h5ad	0.701 s	(0.9%)
rank_genes_groups	14.759 s	(18.4%)
TOTAL	80.087 s	

Per-section Runtime by Dataset

As the number of cells increases, neighbors becomes the main bottleneck and jumps to around 40% of the time. Gene ranking also increased substantially, while UMAP increased but not consistently. Read matrix becomes more significant at large dataset sizes. Startup and argparse remain mostly fixed across datasets. The other functions remain insignificant/not noticeable.

At 20,000 cells, some sections' runtime should remain similar, but others will increase dramatically. Startup_argparse should remain similar because the overhead is mainly fixed. Read_10_mtx runtime will increase slightly because we are processing a larger matrix. PCA will

increase slightly due to higher dimensions. Neighbors and rank_genes_groups serve as the main bottleneck and should double pbmc10k's runtime (takes around 68s and 30s, respectively). UMAP should slightly increase in runtime or be similar to 10k. Louvain and Other should increase slightly. Given the total runtime of pbmc10k is around 80 seconds, a 20,000-cell dataset should take around 145 to 150 seconds to run.

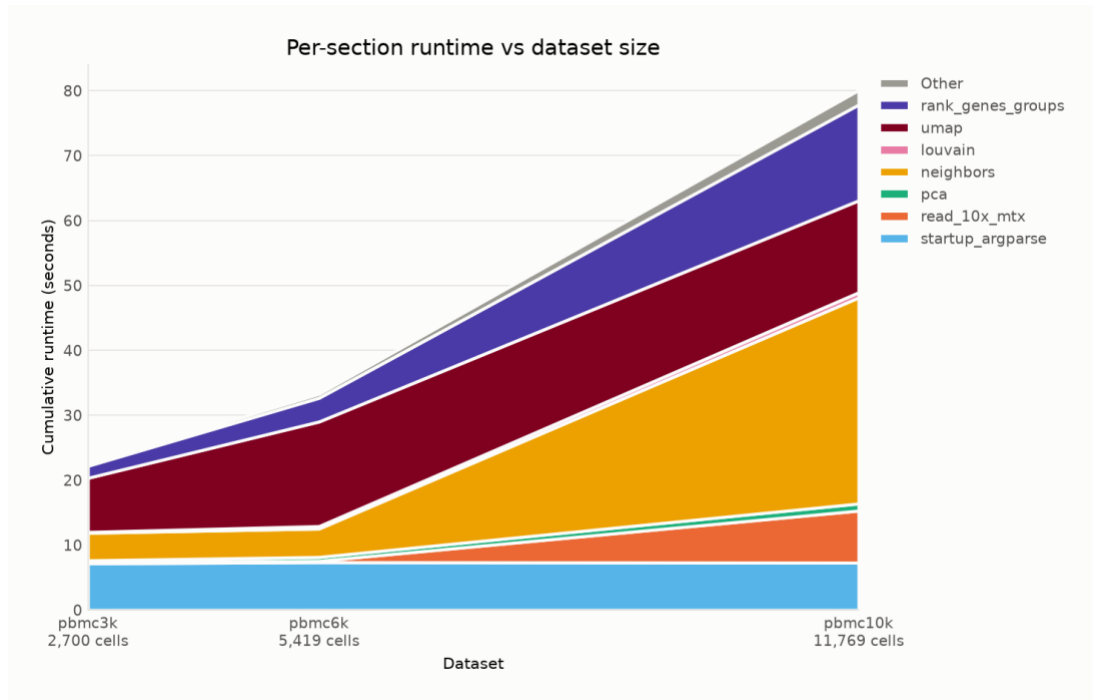


Figure 1 Runtime per Section vs. PBMC datasets (pbmc3k, pbmc6k, and pbmc10k). Each section's runtime is represented by its band thickness.

Fine-grained Profiling (cProfile)

Console output

The original cProfile console outputs contained over 1000 lists for each dataset and would be inappropriate for display. The reduced version (top 50 lists) is available in the Supplemental Materials section.

cProfile Visualizations

The following cProfile visualizations are generated using SnakeViz from the saved .prof files in cProfiling. Separate cProfile runs disabled tqdm's background monitor to prevent inconsistent caller records and produce SnakeViz visualizations. Coarse and instrumented timings were retained from the original experiments.

PBMC3k cProfile

SnakeViz

Reset Root

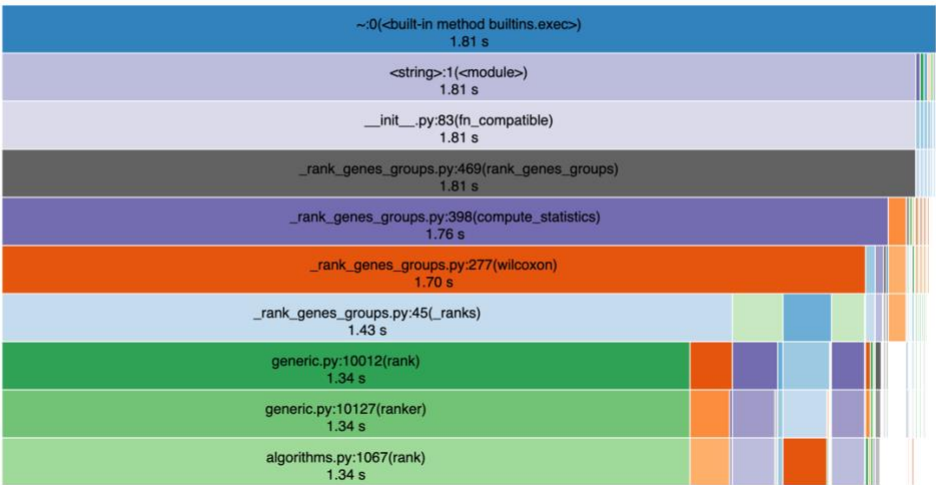
Reset Zoom

Style: Icicle

Depth: 10

Cutoff: 1 / 1000

Call Stack



PBMC6k cProfile

SnakeViz

Reset Root

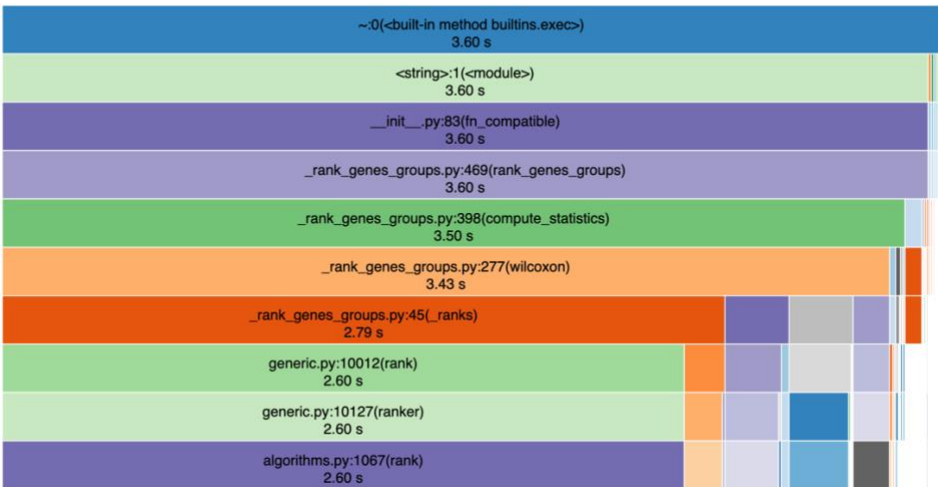
Reset Zoom

Style: Icicle

Depth: 10

Cutoff: 1 / 1000

Call Stack



PBMC10k cProfile

SnakeViz

Reset Root

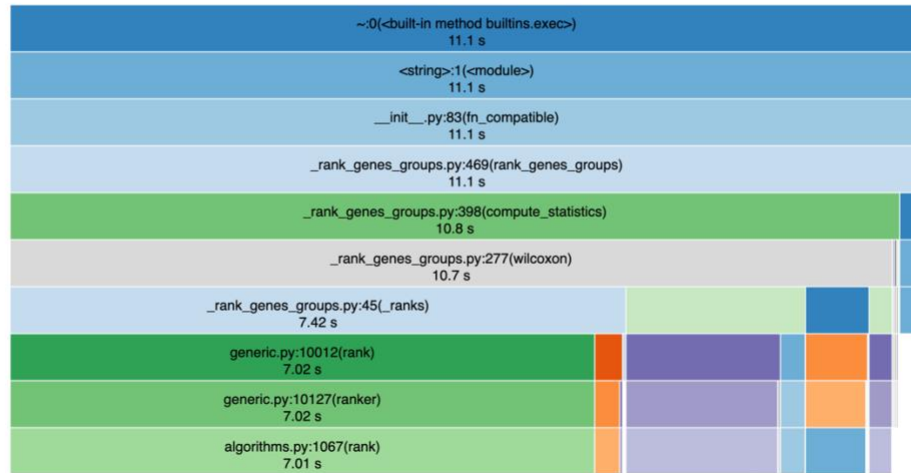
Reset Zoom

Style: **Icicle** ▾

Depth: **10** ▾

Cutoff: **1 / 1000** ▾

Call Stack



SLURM Scripts

Below are the SLURM scripts for running each experiment with the three datasets. Each experiment contains coarse-grained profiling, instrumental profiling, and fine-grained profiling with cProfile.

PBMC3k Slurm Script

```
#!/bin/bash
#SBATCH --job-name=prof-pbmc3k
#SBATCH --partition=batch
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --time=00:30:00
#SBATCH --output=logs/profile_pbmc3k_%j.out
#SBATCH --error=logs/profile_pbmc3k_%j.err
```

```
# Submit: sbatch profile_pbmc3k.sbatch
```

```
set -euo pipefail
```

```
DS=pbmc3k
```

```
HW=/home/rzhou73/BMI500/HW3
```

```

cd "$HW"

mkdir -p logs profiles

source /home/rzhou73/bmi500/bin/activate

# Pin every threading layer to 1 so timings are comparable across datasets
export OMP_NUM_THREADS=1
export OPENBLAS_NUM_THREADS=1
export MKL_NUM_THREADS=1
export NUMBA_NUM_THREADS=1

COARSE="profiles/${DS}_coarse_time.txt"
INSTR="profiles/${DS}_instrumented_time.txt"
FINE="profiles/${DS}_rank_genes_groups_profile.txt"

echo "=====
echo " dataset : $DS"
echo " node   : $(hostname)"
echo " job    : ${SLURM_JOB_ID}"
echo " cpus   : ${SLURM_CPUS_PER_TASK}  mem: 16G"
echo " started : $(date)"
echo "=====

# Extract data if not decompressed yet
if [[ ! -d "data/${DS}/filtered_gene_bc_matrices" ]]; then
    echo "[setup] extracting data/${DS}.tgz"
    tar xzf "data/${DS}.tgz" -C data/
fi

# THE MEASURED RUN
echo
echo "##### RUN: scanpy_pbmc.py under /usr/bin/time -v #####"
/usr/bin/time -v -o "$COARSE" python scanpy_pbmc.py --data-set "$DS"

# ALL THREE EXPERIMENTS
echo
echo "##### 1. COARSE-GRAINED ($COARSE) #####"

```

```
cat "$COARSE"

echo
echo "##### 2. INSTRUMENTED ($INSTR) #####"
cat "$INSTR"

echo
echo "##### 3. FINE-GRAINED ($FINE) #####"
head -20 "$FINE"

echo
echo "finished : $(date)"
ls -lh profiles/${DS}_* 2>/dev/null || true
```

PBMC6k Slurm Script

```
#!/bin/bash
#SBATCH --job-name=prof-pbmc6k
#SBATCH --partition=batch
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=24G
#SBATCH --time=00:45:00
#SBATCH --output=logs/profile_pbmc6k_%j.out
#SBATCH --error=logs/profile_pbmc6k_%j.err

# Profiling run for pbmc6k (5,419 cells)

set -euo pipefail

DS=pbmc6k
HW=/home/rzhou73/BMI500/HW3
cd "$HW"
mkdir -p logs profiles

source /home/rzhou73/bmi500/bin/activate
```

```
# Pin every threading layer to 1 so timings are comparable across datasets
```

```
export OMP_NUM_THREADS=1
```

```
export OPENBLAS_NUM_THREADS=1
```

```
export MKL_NUM_THREADS=1
```

```
export NUMBA_NUM_THREADS=1
```

```
COARSE="profiles/${DS}_coarse_time.txt"
```

```
INSTR="profiles/${DS}_instrumented_time.txt"
```

```
FINE="profiles/${DS}_rank_genes_groups_profile.txt"
```

```
echo "=====
```

```
echo " dataset : $DS"
```

```
echo " node   : $(hostname)"
```

```
echo " job    : ${SLURM_JOB_ID}"
```

```
echo " cpus   : ${SLURM_CPUS_PER_TASK}  mem: 24G"
```

```
echo " started : $(date)"
```

```
echo "=====
```

```
if [[ ! -d "data/${DS}/filtered_gene_bc_matrices" ]]; then
```

```
    echo "[setup] extracting data/${DS}.tgz"
```

```
    tar xzf "data/${DS}.tgz" -C data/
```

```
fi
```

```
# THE MEASURED RUN
```

```
echo
```

```
echo "##### RUN: scanpy_pbmc.py under /usr/bin/time -v #####"
```

```
/usr/bin/time -v -o "$COARSE" python scanpy_pbmc.py --data-set "$DS"
```

```
# ALL THREE EXPERIMENTS
```

```
echo
```

```
echo "##### 1. COARSE-GRAINED ($COARSE) #####"
```

```
cat "$COARSE"
```

```
echo
```

```
echo "##### 2. INSTRUMENTED ($INSTR) #####"
```

```
cat "$INSTR"
```

```
echo
```

```
echo "##### 3. FINE-GRAINED ($FINE) #####"
```

```
head -20 "$FINE"
```

```
echo
```

```
echo "finished : $(date)"
```

```
ls -lh profiles/${DS}_* 2>/dev/null || true
```

PBMC10k Slurm Script

```
#!/bin/bash
```

```
#SBATCH --job-name=prof-pbmc10k
```

```
#SBATCH --partition=batch
```

```
#SBATCH --nodes=1
```

```
#SBATCH --ntasks=1
```

```
#SBATCH --cpus-per-task=4
```

```
#SBATCH --mem=64G
```

```
#SBATCH --time=02:00:00
```

```
#SBATCH --output=logs/profile_pbmc10k_%j.out
```

```
#SBATCH --error=logs/profile_pbmc10k_%j.err
```

```
# Profiling run for pbmc10k
```

```
set -euo pipefail
```

```
DS=pbmc10k
```

```
HW=/home/rzhou73/BMI500/HW3
```

```
cd "$HW"
```

```
mkdir -p logs profiles
```

```
source /home/rzhou73/bmi500/bin/activate
```

```
# Pin every threading layer to 1 so timings are comparable across datasets
```

```
export OMP_NUM_THREADS=1
```

```
export OPENBLAS_NUM_THREADS=1
```

```
export MKL_NUM_THREADS=1
```

```
export NUMBA_NUM_THREADS=1
```

```
COARSE="profiles/${DS}_coarse_time.txt"
```

```
INSTR="profiles/${DS}_instrumented_time.txt"
```

```
FINE="profiles/${DS}_rank_genes_groups_profile.txt"
```

```
echo "=====
```

```
echo " dataset : $DS (11,769 cells)"
```

```
echo " node   : $(hostname)"
```

```
echo " job    : ${SLURM_JOB_ID}"
```

```
echo " cpus   : ${SLURM_CPUS_PER_TASK} mem: 64G"
```

```
echo " started : $(date)"
```

```
echo "=====
```

```
if [[ ! -d "data/${DS}/filtered_gene_bc_matrices" ]]; then
```

```
    echo "[setup] extracting data/${DS}.tgz"
```

```
    tar xzf "data/${DS}.tgz" -C data/
```

```
fi
```

```
# THE MEASURED RUN
```

```
echo
```

```
echo "##### RUN: scanpy_pbmc.py under /usr/bin/time -v #####"
```

```
/usr/bin/time -v -o "$COARSE" python scanpy_pbmc.py --data-set "$DS"
```

```
# ALL THREE EXPERIMENTS
```

```
echo
```

```
echo "##### 1. COARSE-GRAINED ($COARSE) #####"
```

```
cat "$COARSE"
```

```
echo
```

```
echo "##### 2. INSTRUMENTED ($INSTR) #####"
```

```
cat "$INSTR"
```

```
echo
```

```
echo "##### 3. FINE-GRAINED ($FINE) #####"  
head -20 "$FINE"
```

```
echo  
echo "finished : $(date)"  
ls -lh profiles/${DS}_* 2>/dev/null || true
```

Supplemental Materials

Vibe Coding with AI Agent (Prompt and Code)

Prompts with AI

Coding assistant: Claude Code running as the VS Code extension.

LLM: Claude Opus 5 (1M)

Model ID: claude-opus-5[1m]

LLM Prompt 1

****Prompt**:**

Read @matvec_multiply.py and implement the three following functions with comments:

1. A function that compute the dot product of two vectors using a for loop.
2. A function that compute the matrix-vector product using the dot_product function.
3. A main function that tests the matrix-vector product function using randomly generated data of size 1000x1000.

Keep code as concise and possible and prevent excessive function annotations.

****Response**:**

Read matvec_multiply.py

Write matvec_multiply.py (failed: file had changed since last read)

Write concise-code-style.md (memory note)

Write MEMORY.md

Read matvec_multiply.py

The file was reset to the template, so I'll fill it in under your existing prompt comments.

Write matvec_multiply.py

```
import random
```

```
import time
```

```
# create a function to compute the dot product of two vectors using a for loop
```

```
# add comments for the selected function
```

```
def dot_product(a, b):
```

```

# Sum of element-wise products; vectors must be the same length
assert len(a) == len(b), "vectors must have the same length"
result = 0
for i in range(len(a)):
    result += a[i] * b[i]
return result

# create a function to compute the matrix-vector product using the dot_product function
# add comments for the selected function
def matvec_multiply(A, x):
    # Each entry of the result is the dot product of one row of A with x
    return [dot_product(row, x) for row in A]

# create a main function to test the matrix-vector product function using randomly generated data of size 1000x1000
# add comments for the selected function
def main():
    # Random 1000x1000 matrix and length-1000 vector
    n = 1000
    A = [[random.random() for _ in range(n)] for _ in range(n)]
    x = [random.random() for _ in range(n)]

    # Time the product and show a sample of the result
    start = time.time()
    y = matvec_multiply(A, x)
    print(f"Output length: {len(y)}, time: {time.time() - start:.3fs}")
    print(f"First 5 entries: {y[:5]}")

if __name__ == "__main__":
    main()

```

I rewrote matvec_multiply.py to be short, with one-line comments. I didn't run it because you stopped the earlier run.

- dot_product(a, b): checks that the two vectors are the same length, then adds up $a[i] * b[i]$ in a for loop.
- matvec_multiply(A, x): makes a list of dot_product(row, x), one for each row of A.

- main(): builds a random 1000x1000 matrix and a random vector of length 1000, times the product, and prints how long the result is plus its first 5 entries.

You had left import time in the file, so main() uses it for the timing. Your original prompt comments are still above each function. You can run it with `python3 matvec_multiply.py`.

I also saved a note to memory that you want short code without long docstrings.

LLM Prompt 2

****Prompt**:**

Come up with tests that can test the matrix-vector-product function and dot-product function in `matvec_multiply.py`.

****Response**:**

Bash: list project files and show `matvec_tests.py`, `requirements.txt`, `matvec_multiply.py`

Write `matvec_tests.py` (rejected by user, reason: "past the output in the conversation to me here")

Here are the tests for `matvec_tests.py`. I haven't written them to the file or run them. You can run them with `python3 matvec_tests.py`, or with `pytest matvec_tests.py` if `pytest` is installed.

```
import math
```

```
import random
```

```
from matvec_multiply import dot_product, matvec_multiply
```

```
# Check that calling fn(*args) raises an AssertionError
```

```
def raises_assertion(fn, *args):
```

```
    try:
```

```
        fn(*args)
```

```
    except AssertionError:
```

```
        return True
```

```
    return False
```

```
# generate tests for matrix-vector-product function in matvec_multiply.py
```

```

def test_matvec_known_values():
    assert matvec_multiply([[1, 2], [3, 4]], [5, 6]) == [17, 39]


def test_matvec_identity():
    # Identity matrix returns the vector unchanged
    x = [3, -1, 7]
    I = [[int(i == j) for j in range(3)] for i in range(3)]
    assert matvec_multiply(I, x) == x


def test_matvec_non_square():
    # 2x3 matrix times length-3 vector gives length-2 vector
    assert matvec_multiply([[1, 0, 2], [0, 1, -1]], [1, 2, 3]) == [7, -1]


def test_matvec_zero_vector():
    assert matvec_multiply([[1, 2], [3, 4]], [0, 0]) == [0, 0]


def test_matvec_dimension_mismatch():
    # Number of columns must equal vector length
    assert raises_assertion(matvec_multiply, [[1, 2], [3, 4]], [1, 2, 3])


def test_matvec_random_linearity():
    #  $A(x + z)$  should equal  $Ax + Az$ 
    n = 50
    A = [[random.random() for _ in range(n)] for _ in range(n)]
    x = [random.random() for _ in range(n)]
    z = [random.random() for _ in range(n)]
    lhs = matvec_multiply(A, [xi + zi for xi, zi in zip(x, z)])
    rhs = [a + b for a, b in zip(matvec_multiply(A, x), matvec_multiply(A, z))]
    assert all(math.isclose(l, r) for l, r in zip(lhs, rhs))


# generate tests for dot-product function in matvec_multiply.py

```

```

def test_dot_known_values():
    assert dot_product([1, 2, 3], [4, 5, 6]) == 32

def test_dot_orthogonal():
    assert dot_product([1, 0], [0, 1]) == 0

def test_dot_negative_and_float():
    assert math.isclose(dot_product([-1.5, 2.0], [2.0, -0.5]), -4.0)

def test_dot_commutative():
    a = [random.random() for _ in range(100)]
    b = [random.random() for _ in range(100)]
    assert math.isclose(dot_product(a, b), dot_product(b, a))

def test_dot_empty():
    assert dot_product([], []) == 0

def test_dot_length_mismatch():
    assert raises_assertion(dot_product, [1, 2], [1, 2, 3])

# Run all tests without pytest
if __name__ == "__main__":
    tests = [f for name, f in dict(globals()).items() if name.startswith("test_")]
    for test in tests:
        test()
        print(f"PASS {test.__name__}")
    print(f"All {len(tests)} tests passed")

```

What the tests check:

- Matrix-vector product:
- A small example you can check by hand.

- The identity matrix returns the vector unchanged.
- A non-square 2x3 matrix works.
- A zero vector gives all zeros.
- Mismatched sizes raise an error.
- On random data, $A(x+z)$ equals $Ax + Az$, compared with a small tolerance for rounding.
- Dot product:
 - A known result.
 - Perpendicular vectors give 0.
 - Negative and decimal values work.
 - Swapping the two vectors gives the same answer.
 - Two empty vectors give 0.
 - Vectors of different lengths raise an error.

LLM Prompt 3

****Prompt**:**

Given the current implementation of `matvec_multiply` function, assess possible failure mode for the functions and add logic to guard them.

****Response**:**

Read `matvec_multiply.py`

Before changing anything, I'll check how the current code behaves on bad inputs.

Write `scratchpad/probe.py` (tries each bad input and prints the result or error)

Bash: run `probe.py` normally and with `python3 -O` (asserts disabled)

== normal ==

length mismatch	-> AssertionError: vectors must have the same length
None input	-> TypeError: object of type 'NoneType' has no len()
string vectors	-> TypeError: unsupported operand type(s) for +=: 'int' and 'str'
string element	-> TypeError: unsupported operand type(s) for +=: 'int' and 'str'
nested list element	-> TypeError: unsupported operand type(s) for +=: 'int' and 'list'
NaN element	-> returned nan
float overflow	-> returned inf
ragged matrix	-> AssertionError: vectors must have the same length

1-D matrix -> TypeError: object of type 'int' has no len()
None matrix -> TypeError: 'NoneType' object is not iterable
column-vector x -> TypeError: unsupported operand type(s) for +=: 'int' and 'list'
== python -O (asserts disabled) ==
length mismatch -> returned 5
None input -> TypeError: object of type 'NoneType' has no len()
string vectors -> TypeError: unsupported operand type(s) for +=: 'int' and 'str'
string element -> TypeError: unsupported operand type(s) for +=: 'int' and 'str'
nested list element -> TypeError: unsupported operand type(s) for +=: 'int' and 'list'
NaN element -> returned nan
float overflow -> returned inf
ragged matrix -> returned [3, 3]
1-D matrix -> TypeError: object of type 'int' has no len()
None matrix -> TypeError: 'NoneType' object is not iterable
column-vector x -> TypeError: unsupported operand type(s) for +=: 'int' and 'list'

The worst problem: when Python runs with -O, assert statements are skipped, so mismatched sizes return wrong answers without any error. NaN and float overflow also pass through unnoticed. I'll add the guards now.

Write matvec_multiply.py

```
import math
import numbers
import random
import time

# Guard: v must be a list/tuple of finite real numbers
def check_vector(v, name):
    if not isinstance(v, (list, tuple)):
        raise TypeError(f"{name} must be a list or tuple, got {type(v).__name__}")
    for i, val in enumerate(v):
        if not isinstance(val, numbers.Real):
            raise TypeError(f"{name} [{i}] must be a real number, got {type(val).__name__}")
        if not math.isfinite(val):
            raise ValueError(f"{name} [{i}] must be finite, got {val}")
```

```

# create a function to compute the dot product of two vectors using a for loop
# add comments for the selected function
def dot_product(a, b, check=True):
    # Validate inputs; matvec_multiply passes check=False after validating once
    if check:
        check_vector(a, "a")
        check_vector(b, "b")
        if len(a) != len(b):
            raise ValueError(f"vectors must have the same length, got {len(a)} and {len(b)}")

    # Sum of element-wise products
    result = 0
    for i in range(len(a)):
        result += a[i] * b[i]

    # Finite floats can still overflow to inf
    if isinstance(result, float) and not math.isfinite(result):
        raise OverflowError("dot product overflowed")
    return result

# create a function to compute the matrix-vector product using the dot_product function
# add comments for the selected function
def matvec_multiply(A, x):
    # Validate A (list of rows) and x once, so dot_product can skip re-checking x per row
    if not isinstance(A, (list, tuple)) or len(A) == 0:
        raise ValueError("A must be a non-empty list of rows")
    check_vector(x, "x")
    for i, row in enumerate(A):
        check_vector(row, f"A[{i}]")
        if len(row) != len(x):
            raise ValueError(f"A[{i}] has {len(row)} columns but x has length {len(x)}")

    # Each entry of the result is the dot product of one row of A with x
    return [dot_product(row, x, check=False) for row in A]

```

create a main function to test the matrix-vector product function using randomly generated data of size 1000x1000

add comments for the selected function

def main():

Random 1000x1000 matrix and length-1000 vector

n = 1000

A = [[random.random() for _ in range(n)] for _ in range(n)]

x = [random.random() for _ in range(n)]

Time the product and show a sample of the result

start = time.time()

y = matvec_multiply(A, x)

print(f"Output length: {len(y)}, time: {time.time() - start:.3f}s")

print(f"First 5 entries: {y[:5]}")

if __name__ == "__main__":

main()

I've added the guards. Next I'll update the tests: two of them expect the old AssertionError, and I'll add a few tests for the new checks.

Edit matvec_tests.py (replace the raises_assertion helper)

Check that calling fn(*args) raises the given exception type

def raises(exc, fn, *args):

try:

fn(*args)

except exc:

return True

return False

Edit matvec_tests.py (dimension-mismatch test now expects ValueError; add test_matvec_invalid_inputs)

def test_matvec_dimension_mismatch():

Number of columns must equal vector length

assert raises(ValueError, matvec_multiply, [[1, 2], [3, 4]], [1, 2, 3])

```
def test_matvec_invalid_inputs():
    # Ragged rows, empty/None/1-D matrix, and a column-vector x are rejected
    assert raises(ValueError, matvec_multiply, [[1, 2], [3]], [1, 1])
    assert raises(ValueError, matvec_multiply, [], [1, 2])
    assert raises(ValueError, matvec_multiply, None, [1, 2])
    assert raises(TypeError, matvec_multiply, [1, 2], [1, 2])
    assert raises(TypeError, matvec_multiply, [[1, 2]], [[1], [2]])
```

Edit matvec_tests.py (length-mismatch test now expects ValueError; add test_dot_invalid_inputs)

```
def test_dot_length_mismatch():
    assert raises(ValueError, dot_product, [1, 2], [1, 2, 3])
```

```
def test_dot_invalid_inputs():
    # Non-list inputs, non-numeric entries, NaN/inf, and overflow are rejected
    assert raises(TypeError, dot_product, None, [1])
    assert raises(TypeError, dot_product, "ab", [2, 3])
    assert raises(TypeError, dot_product, ["1", 2], [2, 3])
    assert raises(ValueError, dot_product, [float("nan"), 1], [1, 1])
    assert raises(ValueError, dot_product, [float("inf")], [1])
    assert raises(OverflowError, dot_product, [1e200], [1e200])
```

Bash: run python3 matvec_tests.py, python3 -O matvec_tests.py, and python3 matvec_multiply.py

== tests ==

```
PASS test_matvec_known_values
PASS test_matvec_identity
PASS test_matvec_non_square
PASS test_matvec_zero_vector
PASS test_matvec_dimension_mismatch
PASS test_matvec_invalid_inputs
PASS test_matvec_random_linearity
PASS test_dot_known_values
PASS test_dot_orthogonal
```

PASS test_dot_negative_and_float

PASS test_dot_commutative

PASS test_dot_empty

PASS test_dot_length_mismatch

PASS test_dot_invalid_inputs

All 14 tests passed

== tests with -O ==

All 14 tests passed

== main ==

Output length: 1000, time: 0.400s

First 5 entries: [242.61481021538805, 233.95007635917383, 238.10846843577943, 231.89910948926808, 249.68349117187114]

Bash: re-run probe.py with python3 -O

length mismatch -> ValueError: vectors must have the same length, got 2 and 3

None input -> TypeError: a must be a list or tuple, got NoneType

string vectors -> TypeError: a must be a list or tuple, got str

string element -> TypeError: a[0] must be a real number, got str

nested list element -> TypeError: b[0] must be a real number, got list

NaN element -> ValueError: a[0] must be finite, got nan

float overflow -> OverflowError: dot product overflowed

ragged matrix -> ValueError: A[1] has 1 columns but x has length 2

1-D matrix -> TypeError: A[0] must be a list or tuple, got int

None matrix -> ValueError: A must be a non-empty list of rows

column-vector x -> TypeError: x[0] must be a real number, got list

I added input checks to matvec_multiply.py. Every bad input I tried now fails with a clear error instead of a wrong answer. All 14 tests pass, and main() still multiplies the 1000x1000 matrix, taking 0.40s.

How the functions failed before:

- Vectors of different lengths, run with python -O: assert is skipped, so it returned a wrong answer (5) with no error.
- Matrix rows of different lengths, run with python -O: returned a wrong result ([3, 3]) with no error.
- NaN or inf in the input: returned nan with no error.
- Two large floats ($1e200 * 1e200$): returned inf with no error.
- None, strings, or non-numbers inside the vectors: crashed with unclear messages like "unsupported operand type(s) for +=: 'int' and 'str'".

- Matrix that is None, 1-D, or has a column-vector x: crashed with "has no len()" or "not iterable".

What I added:

- `check_vector(v, name)`: a new helper. It requires a list or tuple of real, finite numbers, and the error names the bad position, e.g. "a[0] must be a real number, got str".
- `dot_product`:
 - The assert is now a `ValueError`, so the check still runs under -O.
 - Both vectors are validated.
 - It raises `OverflowError` if a float result becomes inf.
- `matvec_multiply`:
 - Rejects an empty or non-list A.
 - Checks x once and each row once, and says which row has the wrong number of columns.
 - It then calls `dot_product(row, x, check=False)`, so x isn't re-checked for all 1000 rows.

I reran all the bad inputs with `python3 -O`, and each one now raises the expected error.

Changes to `matvec_tests.py`:

- The two size-mismatch tests expected `AssertionError`, so I changed the helper to `raises(exc, fn, *args)`.
- I added `test_matvec_invalid_inputs` and `test_dot_invalid_inputs`, which cover the cases listed above.

Two things to know:

- NumPy arrays: they are now rejected, since only lists and tuples are accepted. Convert them with `.tolist()` first.
- -O and the test file: the tests use `assert` themselves, so run them without -O. The command is still `python3 matvec_tests.py`.

matvec_multiply.py

```
import math
import numbers
import random
import time

# Guard: v must be a list/tuple of finite real numbers
def check_vector(v, name):
    if not isinstance(v, (list, tuple)):
        raise TypeError(f"{name} must be a list or tuple, got {type(v).__name__}")
    for i, val in enumerate(v):
```

```

    if not isinstance(val, numbers.Real):
        raise TypeError(f"{name}[{i}] must be a real number, got {type(val).__name__}")
    if not math.isfinite(val):
        raise ValueError(f"{name}[{i}] must be finite, got {val}")

# create a function to compute the dot product of two vectors using a for loop
def dot_product(a, b, check=True):
    # Validate inputs; matvec_multiply passes check=False after validating once
    if check:
        check_vector(a, "a")
        check_vector(b, "b")
        if len(a) != len(b):
            raise ValueError(f"vectors must have the same length, got {len(a)} and {len(b)}")

    # Sum of element-wise products
    result = 0
    for i in range(len(a)):
        result += a[i] * b[i]

    # Finite floats can still overflow to inf
    if isinstance(result, float) and not math.isfinite(result):
        raise OverflowError("dot product overflowed")
    return result

# create a function to compute the matrix-vector product using the dot_product function
def matvec_multiply(A, x):
    # Validate A (list of rows) and x once, so dot_product can skip re-checking x per row
    if not isinstance(A, (list, tuple)) or len(A) == 0:
        raise ValueError("A must be a non-empty list of rows")
    check_vector(x, "x")
    for i, row in enumerate(A):
        check_vector(row, f"A[{i}]")
        if len(row) != len(x):
            raise ValueError(f"A[{i}] has {len(row)} columns but x has length {len(x)}")

```

```

    # Each entry of the result is the dot product of one row of A with x
    return [dot_product(row, x, check=False) for row in A]

# create a main function to test the matrix-vector product function using randomly generated data of size 1000x1000
def main():
    # Random 1000x1000 matrix and length-1000 vector
    n = 1000
    A = [[random.random() for _ in range(n)] for _ in range(n)]
    x = [random.random() for _ in range(n)]

    # Time the product and show a sample of the result
    start = time.time()
    y = matvec_multiply(A, x)
    print(f"Output length: {len(y)}, time: {time.time() - start:.3f}s")
    print(f"First 5 entries: {y[:5]}")

if __name__ == "__main__":
    main()

```

matvec_tests.py

```

import math
import random

from matvec_multiply import dot_product, matvec_multiply

# Check that calling fn(*args) raises the given exception type
def raises(exc, fn, *args):
    try:
        fn(*args)
    except exc:
        return True
    return False

```

```

# generate tests for matrix-vector-product function in matvec_multiply.py

def test_matvec_known_values():
    assert matvec_multiply([[1, 2], [3, 4]], [5, 6]) == [17, 39]

def test_matvec_identity():
    # Identity matrix returns the vector unchanged
    x = [3, -1, 7]
    I = [[int(i == j) for j in range(3)] for i in range(3)]
    assert matvec_multiply(I, x) == x

def test_matvec_non_square():
    # 2x3 matrix times length-3 vector gives length-2 vector
    assert matvec_multiply([[1, 0, 2], [0, 1, -1]], [1, 2, 3]) == [7, -1]

def test_matvec_zero_vector():
    assert matvec_multiply([[1, 2], [3, 4]], [0, 0]) == [0, 0]

def test_matvec_dimension_mismatch():
    # Number of columns must equal vector length
    assert raises(ValueError, matvec_multiply, [[1, 2], [3, 4]], [1, 2, 3])

def test_matvec_invalid_inputs():
    # Ragged rows, empty/None/1-D matrix, and a column-vector x are rejected
    assert raises(ValueError, matvec_multiply, [[1, 2], [3]], [1, 1])
    assert raises(ValueError, matvec_multiply, [], [1, 2])
    assert raises(ValueError, matvec_multiply, None, [1, 2])
    assert raises(TypeError, matvec_multiply, [1, 2], [1, 2])
    assert raises(TypeError, matvec_multiply, [[1, 2]], [[1], [2]])

def test_matvec_random_linearity():
    #  $A(x + z)$  should equal  $Ax + Az$ 
    n = 50
    A = [[random.random() for _ in range(n)] for _ in range(n)]

```

```

x = [random.random() for _ in range(n)]
z = [random.random() for _ in range(n)]
lhs = matvec_multiply(A, [xi + zi for xi, zi in zip(x, z)])
rhs = [a + b for a, b in zip(matvec_multiply(A, x), matvec_multiply(A, z))]
assert all(math.isclose(l, r) for l, r in zip(lhs, rhs))

# generate tests for dot-product function in matvec_multiply.py
def test_dot_known_values():
    assert dot_product([1, 2, 3], [4, 5, 6]) == 32

def test_dot_orthogonal():
    assert dot_product([1, 0], [0, 1]) == 0

def test_dot_negative_and_float():
    assert math.isclose(dot_product([-1.5, 2.0], [2.0, -0.5]), -4.0)

def test_dot_commutative():
    a = [random.random() for _ in range(100)]
    b = [random.random() for _ in range(100)]
    assert math.isclose(dot_product(a, b), dot_product(b, a))

def test_dot_empty():
    assert dot_product([], []) == 0

def test_dot_length_mismatch():
    assert raises(ValueError, dot_product, [1, 2], [1, 2, 3])

def test_dot_invalid_inputs():
    # Non-list inputs, non-numeric entries, NaN/inf, and overflow are rejected
    assert raises(TypeError, dot_product, None, [1])
    assert raises(TypeError, dot_product, "ab", [2, 3])
    assert raises(TypeError, dot_product, ["1", 2], [2, 3])

```

```

    assert raises(ValueError, dot_product, [float("nan"), 1], [1, 1])
    assert raises(ValueError, dot_product, [float("inf")], [1])
    assert raises(OverflowError, dot_product, [1e200], [1e200])

# Run all tests without pytest
if __name__ == "__main__":
    tests = [f for name, f in dict(globals()).items() if name.startswith("test_")]
    for test in tests:
        test()
        print(f"PASS {test.__name__}")
    print(f"All {len(tests)} tests passed")

```

scanpy_pbmc.py

```

# %%

from typing_extensions import ParamSpecArgs
import numpy as np
import pandas as pd
# import tqdm
# tqdm.tqdm.monitor_interval = 0
import scanpy as sc

import time

_t_start = _t_last = time.time()
_timings = []

def lap(name):
    """Record wall time since the previous lap() call."""
    global _t_last
    now = time.time()
    dt = now - _t_last
    _timings.append((name, dt))
    print(f"[timing] {name:26s} {dt:8.3f} s", flush=True)
    _t_last = now

```

```

import sys
import argparse

# %%
sc.settings.verbosity = 3      # verbosity: errors (0), warnings (1), info (2), hints (3)
sc.logging.print_header()
sc.settings.set_figure_params(dpi=80, facecolor='white')
# sc.settings.n_jobs = int(sys.argv[4])
sc.settings.n_jobs = 1

print(f"using {sc.settings.n_jobs} threads")

# %%
parser = argparse.ArgumentParser(description='Process arguments.')
parser.add_argument('--data-dir', type=str, help='Directory containing the dataset subdirectories', default='data')
parser.add_argument('--data-set', type=str, help='Dataset name, which is the subdirectory name', default='pbmc3k')
parser.add_argument('--out-dir', type=str, help='Output directory', required=False, default='data')
parser.add_argument('--num-threads', type=int, help='Number of threads', default=1, required=False)

args = parser.parse_args()

datadir = args.data_dir if args.data_dir.endswith('/') else args.data_dir + '/'
dataset = args.data_set
outdir = args.out_dir if args.out_dir.endswith('/') else args.out_dir + '/'
nthreads = args.num_threads

lap("startup_argparse")

# %%

# I/O
results_file = "".join([outdir, dataset + '.scanpy.h5ad']) # the file that will store the analysis results

adata = sc.read_10x_mtx(
    #'/nethome/tpan7/scgc/data/' + dataset + '/filtered_gene_bc_matrices/hg19', # the directory with the `.mtx` file
    "".join([datadir, dataset, 'filtered_gene_bc_matrices']), # the directory with the `.mtx` file
    var_names='gene_symbols', # use gene symbols for the variable names (variables-axis index)

```

```

cache=True)                # write a cache file for faster subsequent reading

adata.var_names_make_unique() # this is unnecessary if using `var_names='gene_ids'` in `sc.read_10x_mtx`

lap("read_10x_mtx")

# %%
# preprocessing

# basic filtering
sc.pp.filter_cells(adata, min_genes=200)
sc.pp.filter_genes(adata, min_cells=3)

lap("filter_cells_genes")

# %%
# metric
#adata.var['mt'] = adata.var_names.str.startswith('MT-') # annotate the group of mitochondrial genes as 'mt'
#sc.pp.calculate_qc_metrics(adata, qc_vars=['mt'], percent_top=None, log1p=False, inplace=True)

# filtering by slicing the AnnData object
#adata = adata[adata.obs.n_genes_by_counts < 2500, :]
#adata = adata[adata.obs.pct_counts_mt < 5, :]

# and normalize to 10K reads per cell
sc.pp.normalize_total(adata, target_sum=1e4)
sc.pp.log1p(adata)

lap("normalize_log1p")

# %%
# highly variable genes

#sc.pp.highly_variable_genes(adata, min_mean=0.0125, max_mean=3, min_disp=0.5)
sc.pp.highly_variable_genes(adata, flavor="seurat", n_top_genes=2000)

```

```

# freeze data.
adata.raw = adata

# filtering by highly variable genes.
adata = adata[:, adata.var.highly_variable]

lap("highly_variable_genes")

# %%
# regres out effects of total counts per cell and % mitochondrial genes
# sc.pp.regress_out(adata, ['total_counts', 'pct_counts_mt'])
sc.pp.scale(adata)

lap("scale")

# %%
# report adata - so we can check to see if we are comparable to Seurat
# adata.write(results_file)
# adata

# %%
# pca. parallel via OMP_NUM_THREADS
sc.tl.pca(adata, svd_solver='arpack', n_comps=30)

# adata.write(results_file)
# adata

lap("pca")

# %%
# neighborhood graph
sc.pp.neighbors(adata, n_pcs=30)

lap("neighbors")

# %%
# for fixing disconnected clusters or connectivity issues:

```

```
#sc.tl.paga(adata)

#sc.pl.paga(adata, plot=False) # remove `plot=False` if you want to see the coarse-grained graph
#cs.tl.umap(adata, init_pos='paga')


# adata.write(results_file)
# adata


# %%
# clustering (currently uses leiden, previously using louvain (like Seurat).)
#sc.tl.leiden(adata)
sc.tl.louvain(adata, resolution = 0.5)

lap("louvain")


# %%
# umap
sc.tl.umap(adata, n_components=30)

lap("umap")


# %%
adata.write(results_file)
adata

lap("write_h5ad")


# %%
# support t-test, wilcoxon, logistic regression
# find marker genes
import cProfile
import pstats

#sc.tl.rank_genes_groups(adata, 'louvain', method='wilcoxon', use_raw=True)

import os
```

```

os.makedirs('profiles', exist_ok=True)

# per-dataset filenames
prof_bin = f'profiles/{dataset}_rank_genes_groups.prof'
prof_txt = f'profiles/{dataset}_rank_genes_groups_profile.txt'

cProfile.run(
    "sc.tl.rank_genes_groups(adata, 'louvain', method='wilcoxon', use_raw=True)",
    filename=prof_bin
)

# full readable report -> text file
with open(prof_txt, 'w') as fh:
    pstats.Stats(prof_bin, stream=fh) \
        .strip_dirs().sort_stats('tottime').print_stats(50)

# short version -> console
pstats.Stats(prof_bin) \
    .strip_dirs().sort_stats('tottime').print_stats(10)

lap("rank_genes_groups")

# %%
# Summary for instrumental profiling
total = time.time() - _t_start

lines = ["=== Timing summary per section ===",
        f'{"Function":26s} {"Time":>10s} {"Percentage of total time":>24s}',
        "-" * 64]
lines += [f'{"name":26s} {"dt":8.3f} s ({100*dt/total:5.1f}%)' for name, dt in _timings]
lines.append(f'{"TOTAL":26s} {"total":8.3f} s')

print("\n" + "\n".join(lines))

# saved alongside the coarse-grained and cProfile reports, so the instrumented
# numbers can be re-used later without re-running the pipeline
inst_txt = f'profiles/{dataset}_instrumented_time.txt'

```

```
with open(inst_txt, 'w') as fh:
    fh.write("\n".join(lines) + "\n")
print(f"[profile] wrote {inst_txt}")
```

plot_section_runtime.py

```
#!/usr/bin/env python
"""
Aggregate the instrumented-profiling results and plot per-section runtime
against dataset size as a stacked area chart.

Run: python plot_section_runtime.py
"""

import glob
import os
import re
import tarfile

import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

SURFACE = "#fcfcfb"
INK     = "#0b0b0b"
INK_2   = "#52514e"
GRID    = "#e3e2de"
SLOTS   = ["#56B4E9", "#eb6834", "#1baf7a", "#eda100",
           "#e87ba4", "#800020", "#4a3aa7"]
OTHER   = "#9a9992"

MAX_SERIES = 7      # + "Other" = 8 bands, the categorical cap

# Pipeline sections
```

```

PIPELINE = [
    "startup_argparse", "read_10x_mtx", "filter_cells_genes",
    "normalize_log1p", "highly_variable_genes", "scale", "pca",
    "neighbors", "louvain", "umap", "write_h5ad", "rank_genes_groups",
]

# rows of the instrumented table, e.g. "umap 8.323 s (36.3%)"
ROW_RE = re.compile(r"^(w+)\s+([0-9.]+\s+s)")
SUFFIX = "_instrumented_time.txt"

def n_cells(dataset):
    """Cell count from the MatrixMarket header (line 3: genes cells nnz)."""
    mtx = f"data/{dataset}/filtered_gene_bc_matrices/matrix.mtx"
    if os.path.exists(mtx):
        with open(mtx) as fh:
            return int(fh.readlines()[2].split()[1])
    # not extracted -- stream the header straight out of the tarball
    with tarfile.open(f"data/{dataset}.tgz") as tf:
        fh = tf.extractfile(f"{dataset}/filtered_gene_bc_matrices/matrix.mtx")
        for i, line in enumerate(fh):
            if i == 2:
                return int(line.split()[1])
        raise SystemExit(f"cannot determine cell count for {dataset}")

def load():
    """{dataset: {'n_cells': int, 'sections': {name: seconds}}}"
    out = {}
    for path in sorted(glob.glob(f"profiles/{SUFFIX}")):
        ds = os.path.basename(path)[-len(SUFFIX)]
        sections = {}
        with open(path) as fh:
            for line in fh:
                m = ROW_RE.match(line)
                if m and m.group(1) != "TOTAL":
                    sections[m.group(1)] = float(m.group(2))

```

```

    if sections:
        out[ds] = {"n_cells": n_cells(ds), "sections": sections}
    return out

def fold(data):
    """Keep the MAX_SERIES largest sections; bucket the rest into 'Other'."""
    biggest = max(data, key=lambda d: data[d]["n_cells"])
    ranked = sorted(data[biggest]["sections"],
                     key=lambda s: data[biggest]["sections"][s], reverse=True)
    keep = set(ranked[:MAX_SERIES])
    series = [s for s in PIPELINE if s in keep]      # pipeline order
    folded = {}
    for ds, d in data.items():
        vals = {s: d["sections"].get(s, 0.0) for s in series}
        vals["Other"] = sum(v for k, v in d["sections"].items() if k not in keep)
        folded[ds] = vals
    return series + ["Other"], folded

def main():
    data = load()
    if not data:
        raise SystemExit(f"No profiles/{SUFFIX} found.")

    order = sorted(data, key=lambda d: data[d]["n_cells"])

    series, folded = fold(data)
    colors = [OTHER if s == "Other" else SLOTS[i] for i, s in enumerate(series)]

    cells = np.array([data[d]["n_cells"] for d in order], dtype=float)
    mat = np.array([[folded[d][s] for d in order] for s in series])

    fig, ax = plt.subplots(figsize=(9.5, 6.0), facecolor=SURFACE)
    ax.set_facecolor(SURFACE)

    ax.stackplot(cells, mat, colors=colors, labels=series,

```

```

        edgecolor=SURFACE, linewidth=2) # 2px surface gap between bands

ax.set_xticks(cells)
ax.set_xticklabels([f"{d}\n{int(c):.} cells" for d, c in zip(order, cells)])
ax.set_xlim(cells.min(), cells.max())
ax.set_ylim(0, None)
ax.set_xlabel("Dataset")
ax.set_ylabel("Cumulative runtime (seconds)")

for side in ("top", "right"):
    ax.spines[side].set_visible(False)
for side in ("left", "bottom"):
    ax.spines[side].set_color(GRID)
ax.tick_params(colors=INK_2, labelsize=9.5, length=0)
ax.grid(True, axis="y", color=GRID, linewidth=0.8, alpha=0.9)
ax.set_axisbelow(True)

ax.set_title("Per-section runtime vs dataset size", fontsize=14)

handles, names = ax.get_legend_handles_labels()
ax.legend(handles[:-1], names[:-1], loc="upper left", frameon=False,
          fontsize=9.5, labelcolor=INK_2, bbox_to_anchor=(1.01, 1.0))

fig.tight_layout()
out = "profiles/section_runtime_vs_dataset.png"
os.makedirs("profiles", exist_ok=True)
fig.savefig(out, dpi=150, facecolor=SURFACE, bbox_inches="tight", pad_inches=0.3)
print(f"wrote {out}")

if __name__ == "__main__":
    main()

```

cProfiling Console Output

PBMC3k Console Output (top 50 lines)

Tue Sep 15 15:35:33 2026 profiles/pbmc3k_rank_genes_groups.prof

42155 function calls (40996 primitive calls) in 1.811 seconds

Ordered by: internal time

List reduced from 1031 to 50 due to restriction <50>

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
4	1.337	0.334	1.337	0.334	algorithms.py:1067(rank)
53	0.089	0.002	0.089	0.002	take.py:120(_take_nd_ndarray)
4	0.077	0.019	0.077	0.019	{built-in method
scipy.sparse._sparsetools.csr_todense}					
8	0.076	0.010	0.076	0.010	{built-in method
scipy.sparse._sparsetools.csr_column_index2}					
1	0.035	0.035	0.035	0.035	
__init__.py:781(_check_nonnegative_integers_in_mem)					
270	0.034	0.000	0.034	0.000	{method 'reduce' of
'numpy.ufunc' objects}					
22	0.018	0.001	0.018	0.001	missing.py:261(_isna_array)
5	0.014	0.003	1.435	0.287	
_rank_genes_groups.py:45(_ranks)					
8	0.010	0.001	0.010	0.001	{built-in method
scipy.sparse._sparsetools.csr_column_index1}					
8	0.010	0.001	0.010	0.001	
_utils.py:47(sparse_mean_variance_axis)					
23	0.009	0.000	0.009	0.000	{method 'argsort' of
'numpy.ndarray' objects}					
5	0.006	0.001	1.704	0.341	
_rank_genes_groups.py:277(wilcoxon)					
90	0.005	0.000	0.006	0.000	
numeric.py:1330(normalize_axis_tuple)					
16	0.005	0.000	0.065	0.004	frame.py:11462(_reduce)
4	0.005	0.001	0.005	0.001	{built-in method
scipy.sparse._sparsetools.get_csr_submatrix}					
23	0.004	0.000	0.004	0.000	{built-in method _io.open_code}
99	0.004	0.000	0.004	0.000	{built-in method posix.stat}
328	0.003	0.000	0.003	0.000	{built-in method __new__ of
type object at 0x1500f624dfe0}					
1	0.003	0.003	1.755	1.755	
_rank_genes_groups.py:398(compute_statistics)					
23	0.002	0.000	0.002	0.000	{method '__exit__' of
'_io._IOBase' objects}					
1	0.002	0.002	0.002	0.002	{built-in method
scipy.sparse._sparsetools.csr_eliminate_zeros}					
6276/6249	0.002	0.000	0.004	0.000	{built-in method
builtins.isinstance}					
5	0.002	0.000	0.002	0.000	{built-in method posix.listdir}
92	0.001	0.000	0.001	0.000	base.py:5385(__getitem__)
23	0.001	0.000	0.001	0.000	{built-in method marshal.loads}
20	0.001	0.000	0.002	0.000	multi.py:1194(_engine)
23	0.001	0.000	0.001	0.000	{method 'read' of
'_io.BufferedReader' objects}					
3093/2508	0.001	0.000	0.001	0.000	{built-in method builtins.len}
16	0.001	0.000	0.017	0.001	nanops.py:1499(_maybe_null_out)
45	0.001	0.000	0.007	0.000	function_base.py:5369(insert)
4	0.001	0.000	0.001	0.000	
_continuous_distns.py:370(_norm_cdf)					

52	0.001	0.000	0.002	0.000	multi.py:3001(get_loc)
60	0.001	0.000	0.003	0.000	
construction.py:517(sanitize_array)					
84	0.001	0.000	0.001	0.000	generic.py:6258(__finalize__)
56	0.001	0.000	0.001	0.000	{method 'copy' of
'numpy.ndarray' objects}					
27	0.001	0.000	0.003	0.000	base.py:475(__new__)
79	0.001	0.000	0.001	0.000	managers.py:180(blknos)
40	0.001	0.000	0.001	0.000	{method 'take' of
'numpy.ndarray' objects}					
170	0.001	0.000	0.001	0.000	base.py:650(_simple_new)
20	0.001	0.000	0.001	0.000	
_sputils.py:264(get_index_dtype)					
4	0.000	0.000	0.002	0.000	
_distn_infrastructure.py:2216(sf)					
1049	0.000	0.000	0.001	0.000	generic.py:42(_instancecheck)
94	0.000	0.000	0.001	0.000	generic.py:281(__init__)
200	0.000	0.000	0.000	0.000	{built-in method numpy.empty}
1660	0.000	0.000	0.000	0.000	{built-in method
builtins.getattr}					
5	0.000	0.000	0.003	0.001	records.py:588(fromarrays)
128/5	0.000	0.000	0.001	0.000	{built-in method
_abc._abc_subclasscheck}					
1049	0.000	0.000	0.001	0.000	generic.py:37(_check)
23	0.000	0.000	0.001	0.000	
utils.py:239(maybe_convert_indices)					
4	0.000	0.000	0.005	0.001	multitest.py:99(multipletests)

PBMC6k Console Output (top 50 lines)

Tue Sep 15 15:44:58 2026 profiles/pbmc6k_rank_genes_groups.prof

80020 function calls (78229 primitive calls) in 2.674 seconds

Ordered by: internal time

List reduced from 1031 to 50 due to restriction <50>

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
8	2.024	0.253	2.024	0.253	algorithms.py:1067(rank)
108	0.140	0.001	0.141	0.001	take.py:120(_take_nd_ndarray)
14	0.125	0.009	0.125	0.009	{built-in method
scipy.sparse._sparsetools.csr_column_index2}					
8	0.100	0.012	0.100	0.012	{built-in method
scipy.sparse._sparsetools.csr_todense}					
1	0.045	0.045	0.046	0.046	
__init__.py:781(_check_nonnegative_integers_in_mem)					
693	0.042	0.000	0.042	0.000	{method 'reduce' of
'numpy.ufunc' objects}					
14	0.022	0.002	0.022	0.002	
_utils.py:47(sparse_mean_variance_axis)					
9	0.020	0.002	2.154	0.239	
_rank_genes_groups.py:45(_ranks)					
14	0.020	0.001	0.020	0.001	{built-in method
scipy.sparse._sparsetools.csr_column_index1}					
62	0.019	0.000	0.019	0.000	missing.py:261(_isna_array)

35	0.012	0.000	0.012	0.000	{method 'argsort' of
'numpy.ndarray' objects}					
316	0.006	0.000	0.006	0.000	base.py:650(_simple_new)
8	0.005	0.001	0.005	0.001	{built-in method
scipy.sparse._sparsetools.get_csr_submatrix}					
56	0.005	0.000	0.079	0.001	frame.py:11462(_reduce)
8	0.003	0.000	2.566	0.321	
_rank_genes_groups.py:277(wilcoxon)					
99	0.003	0.000	0.003	0.000	{built-in method posix.stat}
23	0.003	0.000	0.003	0.000	{built-in method _io.open_code}
12783/12753	0.002	0.000	0.004	0.000	{built-in method
builtins.isinstance}					
1	0.002	0.002	0.002	0.002	{built-in method
scipy.sparse._sparsetools.csr_eliminate_zeros}					
632	0.002	0.000	0.002	0.000	{built-in method __new__ of
type object at 0x147436976fe0}					
23	0.001	0.000	0.001	0.000	{method '__exit__' of
'_io._IOBase' objects}					
186	0.001	0.000	0.002	0.000	base.py:5385(__getitem__)
1	0.001	0.001	2.612	2.612	
_rank_genes_groups.py:398(compute_statistics)					
35	0.001	0.000	0.002	0.000	multi.py:1194(_engine)
35	0.001	0.000	0.001	0.000	{built-in method numpy.zeros}
23	0.001	0.000	0.001	0.000	{built-in method marshal.loads}
5783/4587	0.001	0.000	0.001	0.000	{built-in method builtins.len}
56	0.001	0.000	0.023	0.000	nanops.py:1499(_maybe_null_out)
70	0.001	0.000	0.005	0.000	base.py:475(__new__)
7	0.001	0.000	0.001	0.000	
_continuous_distns.py:370(_norm_cdf)					
223	0.001	0.000	0.001	0.000	generic.py:6258(__finalize__)
118	0.001	0.000	0.004	0.000	
construction.py:517(sanitize_array)					
63	0.001	0.000	0.001	0.000	
utils.py:239(maybe_convert_indices)					
120	0.001	0.000	0.001	0.000	{method 'nonzero' of
'numpy.ndarray' objects}					
151	0.001	0.000	0.001	0.000	{method 'copy' of
'numpy.ndarray' objects}					
237	0.001	0.000	0.001	0.000	generic.py:281(__init__)
78	0.001	0.000	0.001	0.000	function_base.py:5369(insert)
2332	0.001	0.000	0.001	0.000	generic.py:42(_instancecheck)
155/154	0.001	0.000	0.162	0.001	indexing.py:1177(__getitem__)
82	0.001	0.000	0.002	0.000	multi.py:3001(get_loc)
56	0.001	0.000	0.001	0.000	
cast.py:123(maybe_convert_platform)					
433	0.001	0.000	0.001	0.000	{built-in method numpy.empty}
83	0.001	0.000	0.001	0.000	{method 'take' of
'numpy.ndarray' objects}					
2332	0.001	0.000	0.001	0.000	generic.py:37(_check)
56	0.001	0.000	0.001	0.000	range.py:1148(take)
7	0.001	0.000	0.002	0.000	
_distn_infrastructure.py:2216(sf)					
64	0.001	0.000	0.144	0.002	
managers.py:642(reindex_indexer)					
7	0.001	0.000	0.007	0.001	multitest.py:99(multipletests)
3370	0.001	0.000	0.001	0.000	{built-in method
builtinsgetattr}					

```
83      0.001      0.000      0.001      0.000
cast.py:1166(maybe_infer_to_datetimelike)
```

PBMC10k Console Output (top 50 lines)

Tue Sep 15 15:40:19 2026 profiles/pbmc10k_rank_genes_groups.prof

323722 function calls (317915 primitive calls) in 11.051 seconds

Ordered by: internal time

List reduced from 1031 to 50 due to restriction <50>

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
24	7.013	0.292	7.015	0.292	algorithms.py:1067(rank)
32	1.566	0.049	1.566	0.049	{built-in method
scipy.sparse._sparsetools.csr_column_index2}					
481	0.695	0.001	0.700	0.001	take.py:120(_take_nd_ndarray)
24	0.298	0.012	0.298	0.012	{built-in method
scipy.sparse._sparsetools.csr_todense}					
32	0.294	0.009	0.294	0.009	
_utils.py:47(sparse_mean_variance_axis)					
32	0.277	0.009	0.277	0.009	{built-in method
scipy.sparse._sparsetools.csr_column_index1}					
1	0.250	0.250	0.251	0.251	
__init__.py:781(_check_nonnegative_integers_in_mem)					
3834	0.129	0.000	0.129	0.000	{method 'reduce' of
'numpy.ufunc' objects}					
25	0.060	0.002	7.416	0.297	
_rank_genes_groups.py:45(_ranks)					
390	0.053	0.000	0.053	0.000	missing.py:261(_isna_array)
71	0.046	0.001	0.046	0.001	{method 'argsort' of
'numpy.ndarray' objects}					
24	0.031	0.001	0.031	0.001	{built-in method
scipy.sparse._sparsetools.get_csr_submatrix}					
384	0.017	0.000	0.271	0.001	frame.py:11462(_reduce)
1	0.014	0.014	0.014	0.014	{built-in method
scipy.sparse._sparsetools.csr_eliminate_zeros}					
17	0.011	0.001	10.656	0.627	
_rank_genes_groups.py:277(wilcoxon)					
56580/56541	0.011	0.000	0.018	0.000	{built-in method
builtins.isinstance}					
1	0.009	0.009	2.164	2.164	
_rank_genes_groups.py:174(_basic_stats)					
2592	0.006	0.000	0.006	0.000	{built-in method __new__ of
type object at 0x1552446f3fe0}					
384	0.005	0.000	0.074	0.000	nanops.py:1499(_maybe_null_out)
407	0.005	0.000	0.027	0.000	base.py:475(__new__)
520	0.005	0.000	0.005	0.000	{method 'nonzero' of
'numpy.ndarray' objects}					
99	0.004	0.000	0.004	0.000	{built-in method posix.stat}
676	0.004	0.000	0.005	0.000	base.py:5385(__getitem__)
20941/15792	0.004	0.000	0.006	0.000	{built-in method
builtins.len}					
1268	0.004	0.000	0.005	0.000	generic.py:6258(__finalize__)

391	0.004	0.000	0.006	0.000	
utils.py:239(maybe_convert_indices)					
16	0.004	0.000	0.004	0.000	
_continuous_distns.py:370(_norm_cdf)					
500	0.004	0.000	0.018	0.000	
construction.py:517(sanitize_array)					
1	0.004	0.004	10.756	10.756	
_rank_genes_groups.py:398(compute_statistics)					
384	0.003	0.000	0.007	0.000	
cast.py:123(maybe_convert_platform)					
1968	0.003	0.000	0.003	0.000	{built-in method numpy.empty}
23	0.003	0.000	0.003	0.000	{built-in method _io.open_code}
1298	0.003	0.000	0.004	0.000	generic.py:281(__init__)
11613	0.003	0.000	0.007	0.000	generic.py:42(_instancecheck)
384	0.003	0.000	0.008	0.000	range.py:1148(take)
856/855	0.003	0.000	0.799	0.001	indexing.py:1177(__getitem__)
392	0.003	0.000	0.712	0.002	
managers.py:642(reindex_indexer)					
80	0.003	0.000	0.005	0.000	multi.py:1194(_engine)
11613	0.003	0.000	0.004	0.000	generic.py:37(_check)
23	0.003	0.000	0.003	0.000	{method '__exit__' of
'_io._IOBase' objects}					
852	0.003	0.000	0.003	0.000	{method 'copy' of
'numpy.ndarray' objects}					
200	0.003	0.000	0.003	0.000	{built-in method builtins.max}
401	0.003	0.000	0.003	0.000	cast.py:550(maybe_promote)
853	0.002	0.000	0.774	0.001	indexing.py:1720(_getitem_axis)
1170	0.002	0.000	0.003	0.000	base.py:650(_simple_new)
23	0.002	0.000	0.002	0.000	{method 'read' of
'_io.BufferedReader' objects}					
78	0.002	0.000	0.002	0.000	{built-in method numpy.zeros}
429	0.002	0.000	0.003	0.000	
cast.py:1166(maybe_infer_to_datetimelike)					
5	0.002	0.000	0.008	0.002	records.py:588(fromarrays)
384	0.002	0.000	0.274	0.001	
generic.py:12524(_min_count_stat_function)					